

OpenAPI Specification 3.1

Guia de Estudio en Espanol — Lo que necesitas saber

Preparacion para Techno Week 8.0 — Banco de Costa Rica

Esta guia cubre los conceptos esenciales de OpenAPI 3.1 que necesitas dominar para tu charla. No es una referencia exhaustiva — se enfoca en lo que te van a preguntar y lo que necesitas explicar con confianza.

1. Que es OpenAPI y por que importa

OpenAPI es un estandar abierto para describir APIs REST. Es un archivo YAML o JSON que define que hace tu API: que endpoints tiene, que parametros acepta, que retorna, y como se autentica. Es el 'plano arquitectonico' de tu API.

Antes se llamaba Swagger Specification. En 2015, Swagger fue donado a la Linux Foundation y renombrado a OpenAPI. Swagger ahora es solo el nombre de las herramientas (Swagger UI, Swagger Editor), no de la especificacion.

Para tu charla:

La spec OpenAPI no es solo documentacion. Es un CONTRATO que puede generar automaticamente: tests de seguridad, validaciones, documentacion interactiva, mocks, y SDKs. Es la fuente de verdad de la que nace todo el pipeline DevSecOps.

2. Estructura basica de un archivo OpenAPI 3.1

Un archivo OpenAPI tiene 5 secciones principales:

SECCION	QUE DEFINE	OBLIGATORIA
openapi	Version de la spec (ej: 3.1.0)	Si
info	Titulo, descripcion, version de la API	Si
servers	URLs donde la API esta disponible	No
paths	Los endpoints y sus operaciones	Si
components	Schemas, security schemes, responses reutilizables	No

Ejemplo minimo completo:

```
openapi: "3.1.0"
info:
  title: SecureBank API
  description: API bancaria para transferencias
  version: "1.0.0"

servers:
- url: https://api.banco.co.cr/v1
  description: Produccion
- url: http://localhost:3000/v1
  description: Desarrollo

paths:
  /accounts/{id}/balance:
    get:
      summary: Consultar saldo
      security:
        - BearerAuth: []
      parameters:
        - name: id
          in: path
          required: true
          schema:
            type: string
```

```
      format: uuid
responses:
  "200":
    description: Saldo consultado
    content:
      application/json:
        schema:
          $ref: "#/components/schemas/Balance"
  "401":
    description: No autenticado

components:
  securitySchemes:
    BearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT
  schemas:
    Balance:
      type: object
      properties:
        available:
          type: number
          format: double
        currency:
          type: string
          enum: [CRC, USD]
```

*Este archivo describe una API con 1 endpoint que requiere JWT y retorna un saldo. Es legible por humanos
Y por maquinas.*

3. Paths y Operaciones

Cada path es un endpoint de la API. Dentro de cada path se definen las operaciones con los metodos HTTP (get, post, put, delete, patch).

Cada operacion puede tener:

ELEMENTO	QUE HACE	EJEMPLO
summary	Descripcion corta	Consultar saldo
description	Descripcion detallada	Retorna el saldo disponible y contable...
operationId	ID unico para la operacion	getAccountBalance
tags	Agrupacion logica	[Accounts]
security	Esquema de autenticacion	[BearerAuth: []]
parameters	Parametros de entrada	path, query, header, cookie
requestBody	Cuerpo del request (POST/PUT)	JSON con datos de transferencia
responses	Respuestas posibles	200, 401, 403, 404, 500

Tipos de parametros:

path: en la URL (/accounts/{id}). query: despues del ? (?page=1). header: en los headers HTTP. cookie: en las cookies.

4. Security Schemes (lo mas importante para tu charla)

Los security schemes definen COMO se autentica la API. Se declaran en components/securitySchemes y se referencian en cada operacion o globalmente. OpenAPI soporta 5 tipos:

TIPO	USO	EJEMPLO BANCARIO
http (bearer)	JWT tokens	App movil del banco envia Bearer token
http (basic)	Usuario + password	Integraciones internas (no recomendado)
apiKey	Key en header/query/cookie	API keys para partners del banco
oauth2	OAuth 2.0 flows	Open Banking: acceso delegado a cuentas
openIdConnect	OpenID Connect	SSO corporativo del banco

Ejemplo: Bearer JWT (el que usas en tu demo):

```
components:
  securitySchemes:
    BearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT
      description: Token JWT con claims sub, role, exp

# Aplicar a un endpoint especifico:
paths:
  /transfers:
    post:
      security:
```

- BearerAuth: []

0 aplicar globalmente a toda la API:

security:

- BearerAuth: []

Si un endpoint NO tiene 'security' definido, Spectral lo detecta como violacion. Esto es exactamente lo que demostramos en la charla.

5. Schemas: tipos de datos y validaciones

Los schemas definen la estructura de los datos que la API maneja. En OpenAPI 3.1, los schemas son 100% compatibles con JSON Schema, lo que permite validaciones muy precisas.

Tipos basicos y formatos:

TIPO	FORMATO	EJEMPLO BANCARIO	VALIDACION
string	uuid	IDs de cuenta	Previene enumeracion (IDOR)
string	date-time	Timestamps	ISO 8601
string	email	Correo del cliente	Validacion de formato
string	pattern	IBAN: CR+20 digitos	Regex personalizado
number	double	Montos de transferencia	Precision decimal
integer	int32	Paginacion	Numeros enteros
boolean	-	Cuenta activa/inactiva	true/false
array	-	Lista de transacciones	items define el tipo

Validaciones disponibles:

VALIDACION	APLICA A	EJEMPLO
minimum / maximum	number, integer	amount: min 0.01, max 50000000
minLength / maxLength	string	description: min 1, max 200
pattern	string	IBAN: ^CR\d{20}\$
enum	string	currency: [CRC, USD]
required	object	Campos obligatorios del request
format	string, number	uuid, date-time, email, double

Ejemplo completo — schema de transferencia bancaria:

```
TransferRequest:
  type: object
  required:
    - source_account_id
    - destination_account_id
    - amount
    - currency
    - description
  properties:
    source_account_id:
      type: string
      format: uuid          # Previene IDOR
    destination_account_id:
      type: string
      format: uuid
    amount:
      type: number
      format: double
      minimum: 0.01         # No permite montos negativos
      maximum: 50000000    # Limite maximo de 50M CRC
    currency:
      type: string
      enum: [CRC, USD]     # Solo monedas permitidas
```

```
description:
  type: string
  minLength: 1      # No puede estar vacia
  maxLength: 200    # Limite de caracteres
```

Cada validacion en la spec es una regla de seguridad. Spectral verifica que esten definidas. El codigo debe implementarlas. ZAP verifica que se cumplan en runtime.

6. Diferencias entre OpenAPI 3.0 y 3.1

ASPECTO	3.0	3.1
JSON Schema	Subconjunto modificado	100% compatible con JSON Schema
Tipos nulos	nullable: true	type: [string, null]
Webhooks	No soportado	Seccion webhooks dedicada
Licencia	Propia de OAI	Apache 2.0 (mas permisiva)
\$ref con siblings	No permitido	Permitido (\$ref + description)
Contenido exclusivo	exclusiveMinimum: true	exclusiveMinimum: valor
Ejemplos	example (singular)	examples (array, JSON Schema)

Lo que debes recordar para la charla:

OpenAPI 3.1 es compatible con JSON Schema estandar. Esto significa que las mismas reglas de validacion que usas en tu spec puedes usarlas para validar requests en runtime. Es un solo lenguaje para definir Y validar.

7. Como la spec genera tests, docs, mocks y validaciones

Este es el corazon de tu charla. La spec no solo documenta, GENERA:

QUE GENERA	HERRAMIENTA	COMO FUNCIONA
Validacion de seguridad	Spectral	Lee la spec y valida reglas (auth en endpoints, HTTPS, etc)
Tests dinamicos	OWASP ZAP	Lee la spec y genera ataques contra cada endpoint
Documentacion interactiva	Swagger UI / Redocly	Genera UI donde se puede probar la API en vivo
Mocks automaticos	Prism (Stoplight)	Genera un servidor mock que responde segun la spec
SDKs de cliente	OpenAPI Generator	Genera codigo cliente en 50+ lenguajes
Validacion de requests	express-openapi-validator	Valida que cada request cumpla la spec en runtime
Contract testing	Dredd / Schemathesis	Verifica que la implementacion cumple la spec

En tu pipeline: Spectral + ZAP usan directamente la spec. No necesitan configuracion extra mas alla del archivo openapi.yaml. La spec ES la configuracion.

8. De donde sale la spec (te lo van a preguntar)

ESCENARIO	COMO SE GENERA	HERRAMIENTAS
Design-First (ideal)	Se escribe ANTES del codigo	Swagger Editor, Stoplight Studio
Code-First (comun)	Se genera desde anotaciones en el codigo	SpringDoc (Java), swagger-jsdoc (Node), FastAPI (Python)
Reverse Engineering	Se infiere de trafico o codigo existente	StackHawk (LLM), Akto, API gateways

Tu respuesta cuando pregunten:

'Para APIs nuevas: design-first, escribir la spec antes del codigo. Para APIs existentes: generarla con anotaciones del framework o con herramientas que la infieren del codigo. Lo importante: una vez que tenes la spec, el pipeline la consume y toda la seguridad se activa.'

9. Referencia rapida para la charla

Si preguntan 'Que es OpenAPI en una frase?'

Es el plano arquitectonico de tu API: describe que hace, que acepta, que retorna, y como se autentica. Legible por humanos y por maquinas.

Si preguntan 'Por que 3.1 y no 3.0?'

3.1 es 100% compatible con JSON Schema estandar. Esto unifica la validacion: las mismas reglas de la spec se usan para validar requests en runtime.

Si preguntan 'Que pasa si la spec y el codigo no coinciden?'

Exactamente para eso existe el pipeline. ZAP ataca la API real y la compara con la spec. Si hay diferencias, falla. Es contract testing automatizado.

Si preguntan 'Esto solo funciona con REST?'

OpenAPI es para REST. Para GraphQL existe la spec GraphQL. Para gRPC existe Protocol Buffers. El concepto de spec-driven aplica a todos, pero las herramientas son diferentes.

Si preguntan 'Donde aprendo mas?'

learn.openapis.org — tutorial oficial. spec.openapis.org — la especificacion completa. swagger.io/docs — guias practicas.

Fuentes: spec.openapis.org/oas/v3.1.0 | learn.openapis.org

Tiempo de estudio: 2-3 horas con esta guia + practicar editando tu openapi.yaml del repo.